Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

Relatório Sistemas Digitais

ELE3515 - Grupo 03 - Problema 03

Líder: Francisco Wellyson Caminha Ferreira
Redator: Mateus Macedo Nogueira
Debatedor: Aysha Dandara Tavres Medeiros

Natal – RN
2026

Resumo

Este trabalho apresenta o projeto e a implementação de uma máquina de troco inteligente na disciplina de Sistemas Digitais. O sistema foi desenvolvido para calcular trocos de forma automática, utilizando a metodologia de projeto RTL. A implementação envolveu elementos combinacionais, como multiplexadores e elementos de memória, como registradores, para armazenamento eprocessamento dos dados. A arquitetura foi organizada em módulos funcionais, incluindo registradores para o valor total da subtração, estado da máquina e quantidade de moedas em cada cofre. Cada módulo foi projetado, simulado e validado individualmente antes da integração ao sistema completo. Os resultados confirmaram o funcionamento correto das operações propostas, evidenciando a viabilidade do projeto e proporcionando uma aplicação prática dos conceitos de circuitos digitais.

Palavras-chave: Maquina de troco; Sistemas Digitais; Projeto RTL.

Sumário

1 Resumo **1**

2 Introdução **4**

3 Fundamentação Teórica **5**

3.1 Metodo RTL 5

3.2 Somador 5

3.3 Subtrator 8

3.4 Comparador de Magnitude 10

3.5 Desenvolvimento do Multiplexador de 8 entradas 12

4 Desenvolvimento da Solução **15**

4.1 MDE de Alto Nível 15

4.2 Dataway 17

4.2.1 Comparador Vtot 17

4.2.2 Contador de Moedas 18

4.2.3 Gerador de Operadores 19

4.2.4 Trocador 20

4.2.5 Verificador de Operações 21

4.2.6 Registradores 22

4.2.7 Relação com o Bloco de Controle 25

4.2.8 Estrutura Geral 26

4.3 MDE de Baixo Nível 28

5 Conclusão **29**

REFERÊNCIAS **30**

APÊNDICE A - Relato semanal **31**

A.1 Equipe **31**

A.2 Defina o problema **31**

A.3 Registro do *brainstorming* **31**

A.4 Pontos-chaves **31**

A.5 Questões de pesquisa **31**

A.6 Planejamento da pesquisa **31**

Lista de Figuras

1 Diagrama do somador de 10 bits 5
2 Diagrama Externo 7
3 Diagrama interno do subtrator de 10 bits. 9
4 Caption 10
5 Diagrama lógico do comparador de magnitude de 10 bits. 12
6 Multiplexador de 4 entradas e 1 bit utilizado como referência. 13
7 Diagrama interno do multiplexador de 8 entradas. 13
8 Diagrama da Máquina de Estados de Alto Nível 16
9 Comparador Vtot 18
10 Contador de Moeda 18
11 Gerador de Operação 19
12 Esquemático do Trocador de Moedas 20
13 Verificador de Operação 22
14 Registrador Vtot 22
15 Registrador Banco Prvisório 23
16 Banco de Referencia 24
17 Registrador LED 25
18 Interface entre o *Dataway* e a máquina de estado. 25
19 Esquemático completo do Dataway 27
20 Esquemático da Máquina de Estados de Baixo Nível 28

Lista de Tabelas

1 Sinais de entrada e saída da máquina de troco. 5
2 Metodologia de projeto RTL. 6
3 Tabela-verdade do somador completo de 1 bit. 6
4 Exemplos de operações do subtrator de 10 bits. 9
5 Condições básicas de comparação entre dois bits. 11
6 Tabela-verdade do multiplexador de 8 entradas. 14
7 Tabela Verdade da Chave Seletora do Trocador 21
8 Variáveis de entrada e saída do *Dataway*. 26
9 Tabela 1 - Identificação da equipe. 31

Introdução

Nos circuitos sequenciais, as saídas não dependem apenas da combinação lógica das entradas atuais, mas também do estado anterior do circuito, o qual pode ser descrito por uma Máquina de Estados Finitos (FSM). Uma FSM é formada pelo conjunto de estados que representa o comportamento de um sistema ao longo do tempo, e esses estados são armazenados em registradores, que dão memória ao circuito. Desse modo, é possível projetar sistemas digitais cujo comportamento varia conforme o estado atual e as condições de entrada.

Nesse contexto, o uso de uma FSM se mostrou adequado ao projeto da máquina de troco, pois o sistema precisa executar diversas etapas de forma sequencial de controle. Cada estado da FSM pode representar uma fase do funcionamento, como a espera por uma solicitação, o carregamento do valor de entrada, a verificação dos cofres disponíveis, o cálculo da quantidade de cada moeda e a liberação das moedas correspondentes. Assim, a FSM garante que o processo ocorra de maneira ordenada e sincronizada com o clock do sistema, além de permitir que novas solicitações sejam ignoradas enquanto um troco está sendo processado.

O problema proposto consiste no projeto de uma máquina de troco que libera, em moedas, um valor determinado colocado em sua entrada. A liberação é realizada por um sistema de cofres, um para cada moeda (R$ 1,00, R$ 0,50, R$ 0,25, R$ 0,10, R$ 0,05 e R$ 0,01), que libera uma moeda sempre que a entrada i_x correspondente está em nível lógico alto e ocorre um pulso de clock. O valor do troco, entre R$ 0,00 e R$ 10,00, é informado por um número (a definir) V e carregado na máquina pelo pulso gerado pelo circuito de um botão sincronizado (BS), cuja entrada é T. A máquina também possui uma saída L, que pisca enquanto o troco é processado, período em que qualquer outra solicitação é ignorada. Além disso, o circuito verifica se algum cofre está vazio ($c_x = 0$) e recalcula o troco utilizando apenas os cofres que ainda possuem moedas. Caso não seja possível trocar o valor de V, a saída L é mantida em nível lógico alto até que um novo valor seja carregado na máquina. Os sinais de entrada e saída, com suas respectivas funções e atribuições na placa DE2, são apresentados na Tabela 1.

Sinal	Tipo	Pino (DE2)	Função
T	Entrada	KEY[3]	Botão sincronizado
V	Entrada	SW (posições pares)	Valor do troco
c	Entrada	SW (posições ímpares)	Indicação de cofre vazio
i	Saída	LEDR (posições ímpares)	Liberação de cada moeda
L	Saída	LEDG[0]	Indicação de processamento
reset	Entrada	KEY[0]	Reset do sistema
clk	Entrada	Clock da placa	Pulso de clock
Return	Saída	LEDG[1]	Indica devolução do dinheiro

Tabela 1: Sinais de entrada e saída da máquina de troco.

Fundamentação Teórica

3.1 Metodo RTL

O projeto em nível de transferência entre registradores (*Register Transfer Level*), ou projeto RTL, engloba diferentes abordagens. De modo geral, o projetista especifica os registradores do circuito, descreve as transferências e operações realizadas sobre os dados de entrada, de saída e dos registradores e define o controle responsável por determinar quando cada operação deve ocorrer.

O ponto inicial de um projeto RTL é a criação de uma máquina de estados de alto nível (HLSM), na qual as condições de transição e as ações dos estados vão além de simples operações booleanas sobre bits de entrada e saída.

Para projetar um circuito sequencial em RTL são necessárias quatro etapas, apresentadas na Tabela 2.

3.2 Somador

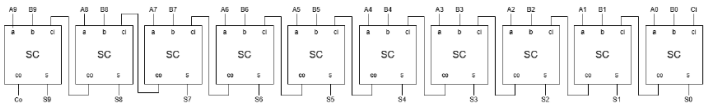


Figura 1: Diagrama do somador de 10 bits

O somador de 10 bits foi desenvolvido a partir da adaptação de um circuito de 8 bits previamente elaborado pelo grupo. Para realizar essa ampliação, foi mantida a estrutura original, composta por oito estágios de soma, sendo acrescentados mais dois estágios para permitir operações com números binários de 10 bits. Dessa forma, o circuito passou a receber duas entradas, A e B, ambas com 10 bits, além de uma entrada de transporte inicial (*carry-in*), responsável por receber o transporte proveniente de uma operação anterior ou um valor lógico previamente definido.

Ordem	Etapa	Descrição
1	Obter uma máquina de estados de alto nível	Descrever o comportamento desejado do sistema na forma de uma máquina de estados de alto nível, composta por estados e transições.
2	Criar um bloco operacional	A partir da máquina de estados de alto nível, criar um bloco operacional capaz de realizar as operações que envolvem dados.
3	Conectar o bloco operacional a um bloco de controle	Conectar o bloco operacional a um bloco de controle, bem como as entradas e saídas booleanas externas ao bloco de controle.
4	Obter a FSM do bloco de controle	Converter a máquina de estados de alto nível na máquina de estados finitos do bloco de controle, substituindo as operações que envolvem dados por sinais de controle, que são ativados ou lidos pelo bloco de controle.

Tabela 2: Metodologia de projeto RTL.

A construção do somador foi baseada na associação de somadores completos de 1 bit em cascata, utilizando a propagação do transporte entre os estágios. Nesse funcionamento, cada bloco realiza a soma dos bits correspondentes das entradas A e B, juntamente com o transporte recebido do estágio anterior. O transporte gerado é encaminhado ao próximo bloco, até alcançar o último estágio. Assim, o circuito produz uma saída S de 10 bits e um sinal de transporte final (*carry-out*).

O funcionamento de cada estágio pode ser representado por meio da tabela-verdade de um somador completo de 1 bit, apresentada na Tabela 3. Nela, são consideradas as combinações possíveis entre as entradas A, B e C_i, bem como os respectivos resultados de soma (S) e transporte (C_o).

Tabela 3: Tabela-verdade do somador completo de 1 bit.

A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

O funcionamento lógico de cada somador completo de 1 bit pode ser descrito por duas expressões booleanas, responsáveis por determinar a soma e o transporte de saída.

A soma é obtida por meio da operação OU-exclusivo (XOR) entre as entradas A, B e C_i, enquanto o transporte de saída é produzido quando pelo menos duas dessas entradas apresentam nível lógico 1. As expressões são apresentadas a seguir:

$$S = A \oplus B \oplus C_i, \tag{1}$$

$$C_o = (A \cdot B) + (A \cdot C_i) + (B \cdot C_i). \tag{2}$$

Nessas expressões, S representa o bit resultante da soma, C_i corresponde ao transporte de entrada e C_o representa o transporte de saída. Como o somador de 10 bits é formado por dez estágios em cascata, essas mesmas expressões são aplicadas individualmente em cada estágio, permitindo a propagação do transporte até o último bloco.

A partir dessa lógica, foi realizada a ampliação do circuito original, adicionando dois novos estágios aos oito já existentes. A Figura 1 apresenta o diagrama interno do somador de 10 bits, destacando a conexão entre os blocos e a propagação do transporte ao longo do circuito.

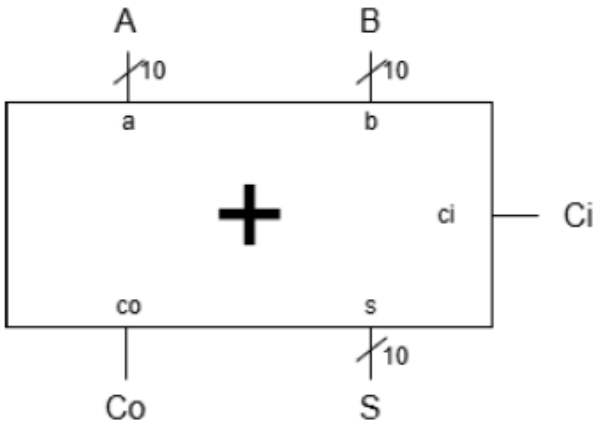


Figura 2: Diagrama Externo

Além da implementação interna, também foi adaptada a representação externa do somador, utilizada para simplificar sua identificação e integração com outros componentes do *Dataway*. Nessa representação, foram mantidas as entradas A e B, a entrada C_i, a saída S e o transporte final C_o, modificando-se os barramentos de dados de 8 para 10 bits. A Figura 2 apresenta o símbolo externo do somador após essa adaptação. O desenvolvimento desse componente é importante para o projeto, pois o somador de 10 bits constitui a base aritmética utilizada na construção do subtrator, permitindo realizar operações de subtração por meio do complemento de dois.

3.3 Subtrator

O subtrator de 10 bits foi desenvolvido com base em um circuito de subtração de 8 bits previamente utilizado pelo grupo. A adaptação consistiu na ampliação dos barramentos de entrada e saída para 10 bits, mantendo-se o princípio de funcionamento por complemento de dois. Para isso, o circuito utiliza o somador de 10 bits desenvolvido anteriormente, um inversor aplicado à entrada B e uma entrada de transporte inicial fixada em nível lógico 1. O funcionamento do subtrator ocorre a partir da inversão dos bits da entrada B, seguida da adição de uma unidade ao resultado. Dessa maneira, a operação de subtração é realizada por meio da seguinte expressão:

$$S = A + \overline{B} + 1 = A - B. \tag{3}$$

A operação de subtração é implementada utilizando o complemento de dois da entrada B. Inicialmente, todos os bits de B são invertidos por meio da operação lógica NOT. Em seguida, adiciona-se uma unidade ao valor invertido, utilizando a entrada de transporte inicial $C_i = 1$ do somador. Dessa forma, a operação $A - B$ é realizada por meio de uma soma binária, aproveitando a estrutura do somador de 10 bits desenvolvido anteriormente.

Para entradas de 10 bits, o resultado é representado pelos dez bits menos significativos da operação. Assim, a saída S corresponde à diferença calculada no intervalo de representação do circuito, enquanto o sinal C_o indica o transporte final produzido pelo somador.

Nesse processo, a entrada A representa o valor do qual será realizada a subtração, enquanto a entrada B corresponde ao valor que será subtraído. O resultado é disponibilizado na saída S, composta por 10 bits, e o circuito também apresenta uma saída de transporte final, denominada C_o.

Para exemplificar o funcionamento do subtrator, a Tabela 4 apresenta algumas operações de subtração entre valores representados em decimal, considerando o intervalo de operação de 10 bits. Esses exemplos demonstram os resultados esperados da operação aritmética realizada pelo circuito.

Tabela 4: Exemplos de operações do subtrator de 10 bits.

A (decimal)	B (decimal)	$S = A - B$ (decimal)
175	100	75
150	50	100
75	25	50
50	10	40
25	5	20
5	1	4
100	100	0

A Figura 3 apresenta o diagrama interno do subtrator de 10 bits, no qual é possível observar a utilização do somador, a inversão da entrada B e a aplicação do valor lógico 1 à entrada C_i, necessários para implementar a operação por complemento de dois.

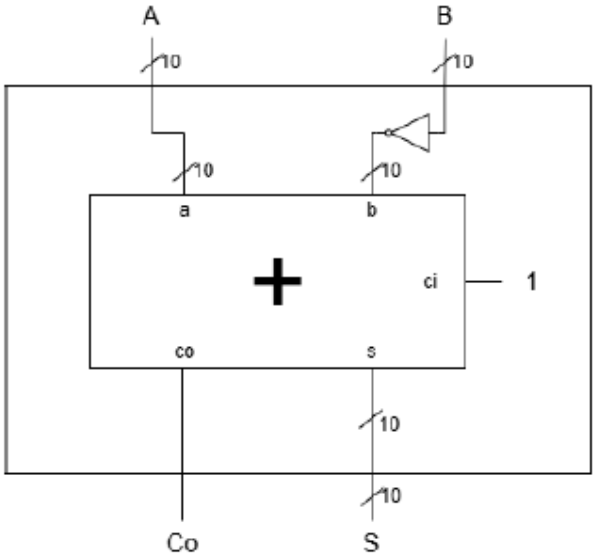


Figura 3: Diagrama interno do subtrator de 10 bits.

Assim como no somador, também foi adaptada a representação externa do subtrator, permitindo sua utilização como um bloco independente nos demais circuitos do projeto. A Figura 4 apresenta essa representação, composta pelas entradas A e B, ambas com 10 bits, pela saída S de 10 bits e pelo sinal de transporte final C_o.

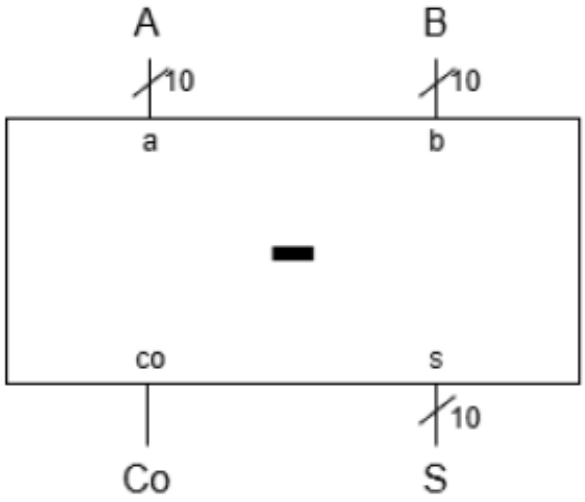


Figura 4: Caption

No projeto da Máquina de Troco, o subtrator possui duas aplicações principais. A primeira ocorre no Trocador, no qual é responsável por diminuir o valor da moeda selecionada do valor restante do troco, representado pela variável V_{tot}. A segunda aplicação ocorre no contador decrescente, no qual pode ser utilizado para reduzir em uma unidade a quantidade provisória de determinada moeda. Dessa forma, o mesmo componente aritmético pode ser aproveitado em diferentes partes do *Dataway*.

3.4 Comparador de Magnitude

O comparador de magnitude de 10 bits foi desenvolvido a partir de um circuito de comparação de 4 bits previamente disponibilizado pelo grupo. Inicialmente, foi considerada a adaptação de um comparador formado por três blocos de 4 bits. Entretanto, optou-se por uma estrutura mais simples, baseada na utilização de dez blocos comparadores de 1 bit conectados em sequência, seguindo o modelo de propagação de sinais empregado no circuito original.

Para a construção do novo comparador, um dos blocos de comparação de 1 bit foi utilizado como referência e replicado até formar dez estágios. Os blocos foram organizados da esquerda para a direita, iniciando pela comparação dos bits mais significativos, A_9 e B_9, e terminando nos bits menos significativos, A_0 e B_0. Cada estágio possui duas entradas de dados, identificadas como a e b, além de três entradas de comparação, denominadas *In_ gt*, *In_ eq* e *In_ lt*, e três saídas correspondentes, *Out_ gt*, *Out_ eq* e *Out_ lt*.

O princípio de funcionamento de um comparador de 1 bit consiste em identificar as relações de igualdade, maior ou menor entre os valores binários recebidos. A Tabela 5 apresenta as condições básicas de comparação entre as entradas A e B, utilizadas como referência para compreender o funcionamento de cada estágio.

Tabela 5: Condições básicas de comparação entre dois bits.

A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

O funcionamento básico do comparador de magnitude de 1 bit pode ser representado por três expressões booleanas, responsáveis por identificar as relações de maior, igual e menor entre as entradas A e B. A condição de maior ocorre quando A apresenta nível lógico 1 e B apresenta nível lógico 0. A condição de igualdade ocorre quando ambas as entradas possuem o mesmo valor lógico, enquanto a condição de menor ocorre quando A apresenta nível lógico 0 e B apresenta nível lógico 1. As expressões correspondentes são:

$$G = A \cdot \overline{B}, \tag{4}$$

$$E = \overline{A \oplus B}, \tag{5}$$

$$L = \overline{A} \cdot B. \tag{6}$$

Nessas expressões, G representa a condição A maior que B, E indica a igualdade e L representa a condição A menor que B.

Para realizar a comparação de números binários de 10 bits, essas condições são utilizadas em conjunto com os sinais de propagação entre os estágios. Considerando uma estrutura em que a decisão dos bits mais significativos possui prioridade, as expressões de propagação podem ser representadas por:

$$Out_{gt} = In_{gt} + (In_{eq} \cdot G), \tag{7}$$

$$Out_{eq} = In_{eq} \cdot E, \tag{8}$$

$$Out_{lt} = In_{lt} + (In_{eq} \cdot L). \tag{9}$$

Dessa maneira, uma condição de maior ou menor identificada em um bit mais significativo é preservada nos estágios seguintes. Quando os bits comparados são iguais, a decisão permanece dependente dos próximos bits, permitindo determinar a relação de magnitude entre os dois números de 10 bits.

Na estrutura proposta, as saídas de comparação de cada bloco são conectadas às entradas correspondentes do estágio seguinte, permitindo a propagação das informações de magnitude ao longo do circuito. As condições iniciais são estabelecidas como $In_gt = 0$, $In_eq = 1$ e $In_lt = 0$, representando uma condição inicial de igualdade. Ao final da sequência, são obtidas três saídas responsáveis por indicar se A é maior que B ($AgtB$), igual a B ($AeqB$) ou menor que B ($AltB$).

A Figura 5 apresenta o diagrama do comparador de magnitude de 10 bits, destacando a organização dos dez blocos em cascata e as conexões entre suas entradas e saídas de comparação.

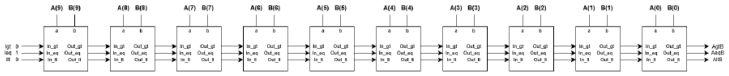


Figura 5: Diagrama lógico do comparador de magnitude de 10 bits.

No projeto da Máquina de Troco, esse comparador permite verificar relações de magnitude entre valores binários de 10 bits, auxiliando nas decisões relacionadas ao valor restante do troco e aos valores das moedas. A utilização das saídas de comparação depende das condições de operação estabelecidas pelo sistema de controle.

3.5 Desenvolvimento do Multiplexador de 8 entradas

O desenvolvimento do multiplexador de 8 entradas teve como referência um circuito de 4 entradas e 1 bit disponibilizado pelo grupo. O circuito original era composto por três multiplexadores de duas entradas, organizados em dois níveis, permitindo selecionar uma entre quatro entradas de dados por meio de dois sinais de seleção.

Para ampliar essa estrutura, foi adotado o mesmo princípio de associação de multiplexadores de duas entradas, acrescentando novos blocos até formar uma árvore composta por sete multiplexadores, distribuídos em três níveis. Essa organização permite selecionar uma entre oito entradas de dados utilizando três sinais de controle.

No primeiro nível, foram utilizados quatro multiplexadores responsáveis pela seleção entre os pares de entradas I_0 e I_1, I_2 e I_3, I_4 e I_5, e I_6 e I_7. Esses blocos são controlados pelo sinal S_2. No segundo nível, dois multiplexadores recebem os resultados das seleções anteriores e são controlados pelo sinal S_1. Por fim, no terceiro nível, um único multiplexador recebe as duas saídas do segundo nível e utiliza o sinal S_0 para determinar o valor encaminhado à saída final D.

A Figura 6 apresenta o circuito original de 4 entradas utilizado como referência para o desenvolvimento do novo multiplexador.

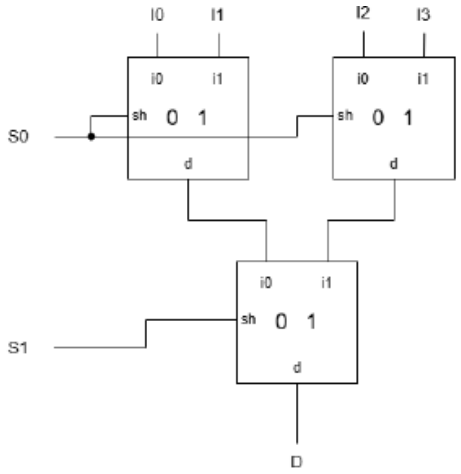


Figura 6: Multiplexador de 4 entradas e 1 bit utilizado como referência.

A Figura 7 apresenta a estrutura proposta para o multiplexador de 8 entradas, destacando os sete blocos de seleção e suas respectivas conexões.

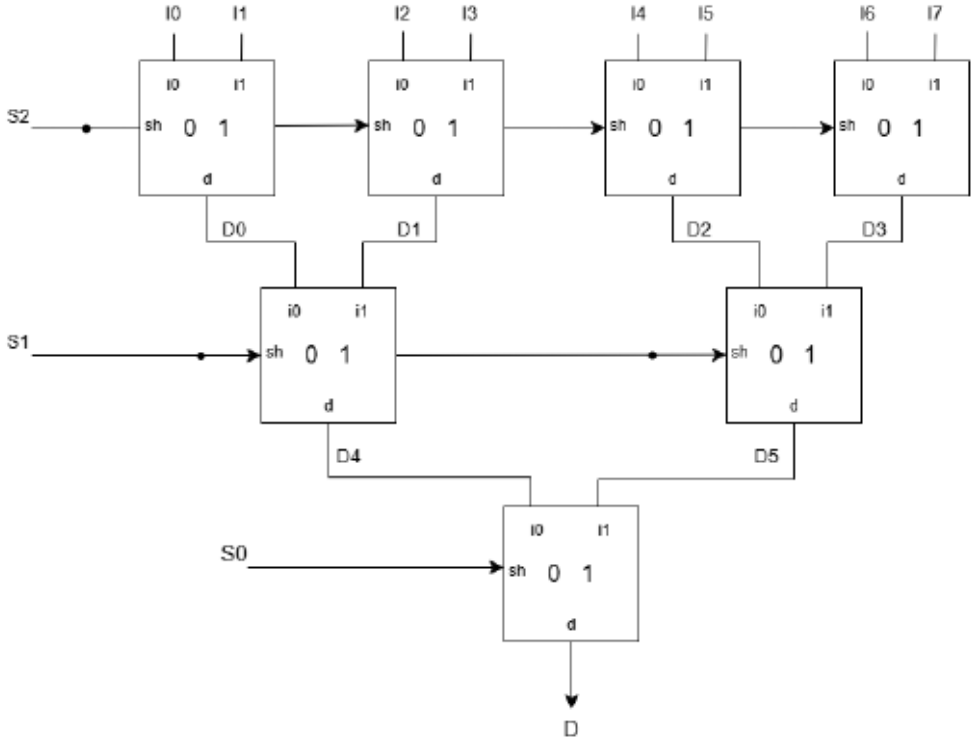


Figura 7: Diagrama interno do multiplexador de 8 entradas.

O funcionamento da estrutura proposta é apresentado na Tabela 6, que relaciona as combinações dos sinais de seleção à entrada encaminhada à saída.

Tabela 6: Tabela-verdade do multiplexador de 8 entradas.

S_0	S_1	S_2	Saída D
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

O funcionamento do multiplexador de 8 entradas é baseado na associação de sete multiplexadores de 2 entradas, organizados em três níveis. Cada multiplexador de 2 entradas possui duas entradas de dados, I_0 e I_1, um sinal de seleção Sel e uma saída D. Seu funcionamento pode ser representado pela seguinte expressão booleana:

$$D = (\overline{Sel} \cdot I_0) + (Sel \cdot I_1). \tag{10}$$

Essa expressão estabelece que, quando Sel apresenta nível lógico 0, a entrada I_0 é encaminhada à saída. Quando Sel apresenta nível lógico 1, a entrada selecionada é I_1.

No multiplexador de 8 entradas, essa operação é realizada sucessivamente em três níveis. O primeiro nível utiliza o sinal S_2 para selecionar entre quatro pares de entradas, o segundo utiliza S_1 para reduzir os quatro resultados a dois, e o terceiro utiliza S_0 para determinar a saída final.

Considerando essa organização, a expressão booleana da saída D pode ser representada por:

$$\begin{aligned} D = {} & \overline{S_0}\,\overline{S_1}\,\overline{S_2}\,I_0 + \overline{S_0}\,\overline{S_1}\,S_2\,I_1 + \overline{S_0}\,S_1\,\overline{S_2}\,I_2 + \overline{S_0}\,S_1\,S_2\,I_3 \\ & + S_0\,\overline{S_1}\,\overline{S_2}\,I_4 + S_0\,\overline{S_1}\,S_2\,I_5 + S_0\,S_1\,\overline{S_2}\,I_6 + S_0\,S_1\,S_2\,I_7. \end{aligned} \tag{11}$$

Dessa forma, a expressão demonstra como os sinais de seleção determinam qual entrada será encaminhada à saída. Para a aplicação no Trocador, essa mesma estrutura de seleção pode ser utilizada com barramentos de 10 bits, permitindo selecionar os valores das moedas representados em binário.

No projeto da Máquina de Troco, essa estrutura será utilizada como referência para a seleção dos valores das moedas. Para essa aplicação, será necessário adaptar a largura das entradas e da saída para 10 bits, permitindo representar os valores monetários utilizados pelo sistema. A saída do multiplexador será encaminhada ao subtrator do Trocador, responsável por atualizar o valor restante do troco.

Desenvolvimento da Solução

A máquina de troco proposta realiza o troco do dinheiro fornecido à maquina pela entrada com nome de V utilizando as moedas de R$1.00, R$0.50, R$0.25, R$0.10, R$0.05 e R$ 0,01. A ordem de prioridade para o troco com essas moedas funciona de forma decrescente, de maneira que sempre tentará realizar o troco primeiro com a moeda de um real, se estiver disponível.

O maquinário conta com um banco de moedas interno separado para cada tipo. Cada vez que uma moeda é utilizada para o troco, o banco reduz sua quantia armazenada. A estrutura conta com alavancas das quais desabilitam ou habilitam o uso de algum tipo de moeda para realizar o troco, ainda que existam moedas dentro do banco. O banco conta com um total máximo de 15 moedas de cada tipo para realizar suas operações antes de necessitar reabastecer.

O sistema realiza o troco fornecendo moedas uma por vez, no entanto essas moedas não são liberadas imediatamente para o usuário. Somente quando o troco foi concluído com sucesso com as moedas disponíveis no momento da requisição do troco que são liberadas as moedas para o usuário. Em caso de erro com o troco, as moedas utilizadas são devolvidas à máquina sem desperdício e o dinheiro depositado ao usuário é devolvido para ele.

A maneira utilizada para implementar esse sistema de troco que só é liberado após terminar todo o processamento utiliza registradores para guardar a quantidade de moedas presente no banco, mais precisamente dois registradores para cada banco. Um registrador é utilizado para guardar o valor de referência da quantidade de moedas enquanto o outro registrador possui um valor provisório que decresce conforme cada troco é efetuado.

Dessa forma, a máquina de troco proposta funciona completamente, ou seja sem fornecer ao usuário valores de troco incompletos ou devorar o dinheiro investido, dando mais satisfação ao cliente. No entanto, a máquina é por sua vez mais cara, devido ao duplo uso de registradores para registrar a quantidade de cada moeda, bem como possui uma resposta lenta a erros, somente informando o erro após todo o processamento que, por ir depositando visivelmente moedas um a um, traz uma perda de tempo e incerteza ao usuário.

O projeto da máquina descrita acima utilizou o método RTL, sendo descrito nos tópicos seguintes.

4.1 MDE de Alto Nível

A máquina de estados de alto nível que modela a máquina de troco desenvolvida encontra-se no esquemático abaixo:

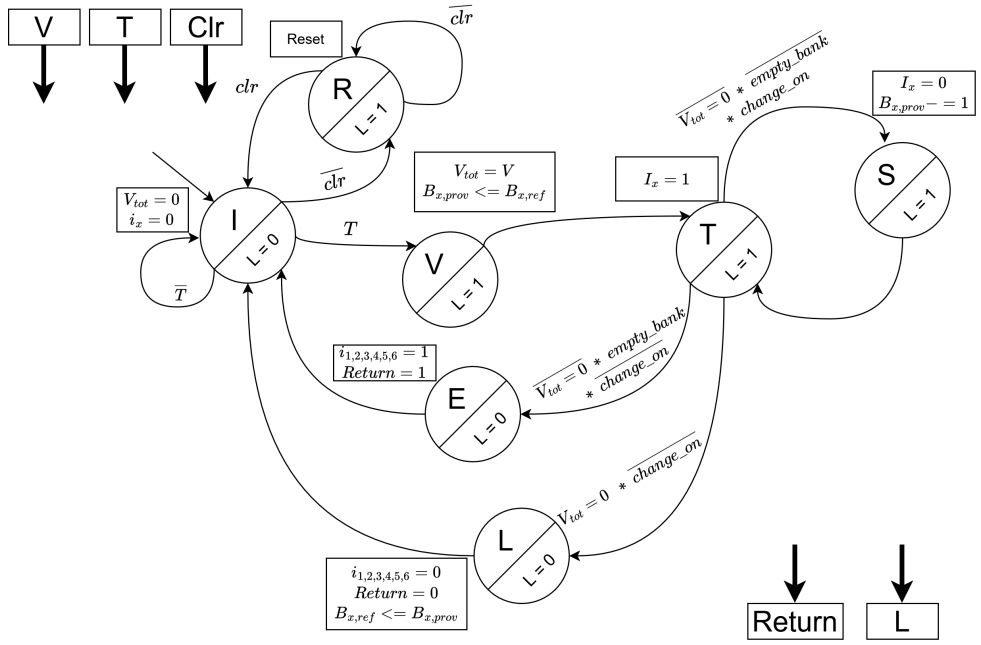


Figura 8: Diagrama da Máquina de Estados de Alto Nível

O sistema conta com um total de 7 estágios. Seu princípio de funcionamento consiste em esperar o input T com o valor binário de V para iniciar o processamento do troco. Durante esse processamento o sistema fica em um loop entre os estados T e S. Após todo o processamento o sistema pode ir para o estado de L, onde as moedas são liberadas para o usuário, ou para o estado de erro, E, em que o dinheiro de input é devolvido - led de devolução torna-se ativo. Depois do resultado, o sistema retorna ao estado inicial. E a partir desse estado inicial é possível reinicializar as variáveis do sistema.

A máquina de estados conta com três entradas externas, sendo elas dois botões/chaves da FPGA de um bit, T e Clr, e a ultima contém 10 bits. Essa entrada maior indica o valor a ser trocado, em unidade de centavos e aceita inputs de até R$ 10,23, enquanto T é o botão utilizado para iniciar o processamento do troco e Clr é a chave para reinicializar o sistema.

Ela também possui duas saídas externas, L e Return. Ambos são leds de um bit que indicam que o sistema está processando e que o dinheiro de entrada será devolvido ao dono. O sinal L ficará sempre ligado enquanto o sistema está processando ou enquanto o sistema está resetando. Ao passo que Return pulsará uma vez somente quando for um indicativo de erro do sistema.

O estado I é o estado inicial do sistema e pós-processamento de um troco, em que todos os LEDs estão desligados e valor sendo trocado no sistema é nulo. Ele espera o sinal ativo de T para trocar para o estado V e se mantem nesse estado enquanto o botão não for acionado. Uma outra alternativa é reinciar os valores do sistema com o acionar de Clr.

O estado R é o estado em que todos os registradores resetam para seus valores padrões, sendo o processo de reabastecimento das moedas após seções de trocos concluídos.

O sistema permanece nesse estado até que o usuário solte o botão, para então voltar ao estado inicial.

O estado V é um estado intermediário utilizado para inicializar o processamento do troco, fazendo o banco provisório receber o valord o banco de referência e o registrador de Vtot inicializar com o valor de V. Ele é necessário pois esses valores serão lidos e modificados nos estados seguintes.

O estado T é o estado que inidica que houve um troco e acende o led da moeda correspondente. Ele lê o valor atual de Vtot e obtem os dados do dataway para saber qual dos outros três estados ele vai seguir. O fluxo comum é ele continuar no loop (indo para o estado S) enquanto estiver operando e não houver erro. O indicativo de erro nesse sistema é se o banco de moedas está vazio mas o valor trocado não chegou a zero.

O estado S é o estado auxiliar ao estado de troca, trabalhando em dupla com o T formando o loop. É o responsável por atualizar os valores de Vtot e do banco do tipo de moeda utilziado pois já ocorreu o troco no estado passado e deve ser salvo os novos valores para ser verificado se ocorre um troco ou não no estado de Troco.

O estado L é o estado de liberação das moedas para o usuário, em que o troco foi um sucesso e portanto o registrador de referência deve ser atualizado com o valor atual do banco provisório. Como o troco para V acabou, volta-se para o estado inicial.

O estado E é o estado de erro, indicando que as moedas disponíveis para uso nos bancos não foi suficiente ou adequado para realizar troco. Logo, deve-se retornar o dinheiro de entrada e acende-se todos os leds para indicar claramente a falha. Depois retorna-se ao estado I para esperar um novo ciclo.

4.2 Dataway

O bloco de controle de dados é responsável por informar à máquina de estados os elementos necessários para a transição dos estados, realizando as operações lógicas e atualizando os registradores internos conforme neceesário. Apresenta-se em sequência os componentes utilziados pelo dataway para implementar essa lógica e depois sua estrura completa.

4.2.1 Comparador Vtot

O comparador de V_{tot} é responsável por comparar, ao mesmo tempo, o valor restante do troco com o valor de cada moeda, fornecendo à lógica de seleção as informações necessárias para decidir qual moeda liberar. Ele é formado por sete comparadores de magnitude de 10 bits, todos ligados em paralelo à mesma variável V_{tot}, que é aplicada à primeira entrada de cada um. A segunda entrada de cada comparador recebe uma constante binária de 10 bits correspondente a um valor em centavos: 0001100100 (100) no comparador de R$ 1,00, 0000110010 (50) no de R$ 0,50, 0000011001 (25) no de R$ 0,25,

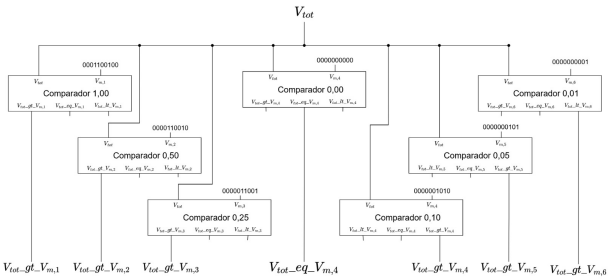


Figura 9: Comparador Vtot

0000001010 (10) no de R$ 0,10, 0000000101 (5) no de R$ 0,05 e 0000000001 (1) no de R$ 0,01. O sétimo comparador recebe a constante 0000000000 e serve para detectar o fim do troco. Cada comparador possui três saídas, de maior, de igual e de menor, mas apenas as necessárias são utilizadas: nos comparadores de moedas, a saída de maior ($V_{tot_gt_V_{m,i}}$) indica que o valor restante supera o valor da moeda correspondente e, portanto, que essa moeda pode ser liberada; já no comparador com a constante zero, a saída de igual indica que $V_{tot} = 0$, ou seja, que o troco foi concluído, e corresponde ao sinal $VTOT_EQ_0$ enviado à máquina de estado. Como os sete comparadores são puramente combinacionais e operam simultaneamente, o resultado de todas as comparações fica disponível em um único ciclo, sem etapas intermediárias.

4.2.2 Contador de Moedas

O bloco possui 12 entradas do tipo std_logic e 7 saídas de também um bit, conforme pode ser observado na imagem abaixo:



Figura 10: Contador de Moeda

Este componente informa se há a existência de moedas dentro do banco (registrador) respectivo a cada moeda e se esse tipo de moeda está disponível para ser utilizada no troco, visto que o usuário pode desabilitar o uso de uma moeda com as chaves C_x. As

expressões lógicas que expressam a saída encontram-se abaixo:

$$has_M_1 = B_{m,1}_gt_0 \; and \; C_1$$
$$has_M_2 = B_{m,2}_gt_0 \; and \; C_2$$
$$has_M_3 = B_{m,3}_gt_0 \; and \; C_3$$
$$has_M_4 = B_{m,4}_gt_0 \; and \; C_4$$
$$has_M_5 = B_{m,5}_gt_0 \; and \; C_5$$
$$has_M_6 = B_{m,6}_gt_0 \; and \; C_6$$

O banco também informa se o banco está vazio por meio da saída *empty*, conforme a expressão em sequência:

$$empty = has_M_1 \; nor \; has_M_2 \; nor \; has_M_3 \; nor \; has_M_4 \; nor \; has_M_5 \; nor \; has_M_6$$

4.2.3 Gerador de Operadores

O gerador de operadores informa qual tipo moeda estará sendo uitlizado para realizar o troco nesse pulso de relógio, pois o sistema troca uma moeda por vez acendendo seu respectivo LED. Ele possui 12 entradas e 7 saídas de um bit, como apresentado na figura abaixo:



Figura 11: Gerador de Operação

Internamente, o bloco realiza a operação lógica AND entre a entrada que indica a presença de moedas válidas nos bancos com a entrada que indica se o valor de V sendo trocado, (V_{tot}, é maior do que o valor atual da moeda (que indica que deve ser feito o troco de uma moeda daquele tipo.

Como só pode haver uma única moeda sendo trocada por vez, as moedas seguintes ao valor de um real só devem ser ativas quando as anteriores, de maior valor, estarem em

nível lógico baixo, sendo modelado pelas expressões:

$$
\begin{aligned}
M_{1,on} &= V_{tot}_gt_V_{m,1}\ and\ has_M_1 \\
M_{2,on} &= V_{tot}_gt_V_{m,2}\ and\ has_M_2\ and\ notM_{1,on} \\
M_{3,on} &= V_{tot}_gt_V_{m,3}\ and\ has_M_3\ and\ notM_{1,on}\ and\ notM_{2,on} \\
M_{4,on} &= V_{tot}_gt_V_{m,4}\ and\ has_M_4\ and\ notM_{1,on}\ and\ notM_{2,on} \\
&\quad and\ notM_{3,on} \\
M_{5,on} &= V_{tot}_gt_V_{m,5}\ and\ has_M_5\ and\ notM_{1,on}\ and\ notM_{2,on} \\
&\quad and\ notM_{3,on}\ and\ notM_{4,on} \\
M_{6,on} &= V_{tot}_gt_V_{m,6}\ and\ has_M_6\ and\ notM_{1,on}\ and\ notM_{2,on} \\
&\quad and\ notM_{3,on}\ and\ notM_{4,on}\ and\ notM_{5,on}
\end{aligned}
$$

Além disso, o componente também diz se alguma operação está acontecendo através da saída *change_on*, que tem seu valor conforme os níveis lógicos das expressões anteriores em uma porta OR de seis entradas uma saída, conforme a expressão lógica em sequência:

$$change_{on} = M_{1,on}\ or\ M_{2,on}\ or\ M_{3,on}\ or\ M_{4,on}\ or\ M_{5,on}\ or\ M_{6,on}$$

4.2.4 Trocador

Este bloco é o responsável por calcular a subtração do valor sendo trocado pelo valor do tipo de moeda realizando o troco no momento. A estrutura do componente encontra-se na figura abaixo:

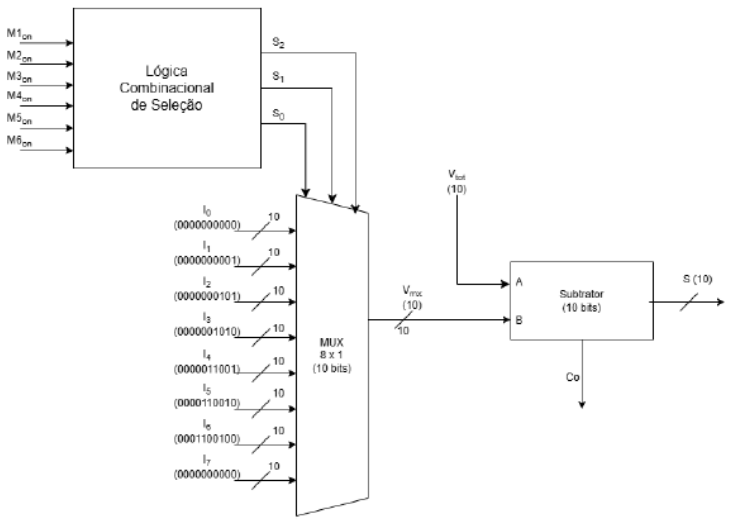


Figura 12: Esquemático do Trocador de Moedas

A estrutura interna consiste de um subtrator de 10 bits que subtrai a entrada V_{tot} com o resultado da seleção de um multiplexador de 8 entradas que seleciona entre os valores das moedas.

A forma de selecionar cada entrada vem através de um bloco de lógica combinacional auxiliar que implementa a tabela abaixo:

Tabela 7: Tabela Verdade da Chave Seletora do Trocador

$M_{1,on}$	$M_{2,on}$	$M_{3,on}$	$M_{4,on}$	$M_{5,on}$	$M_{6,on}$	S2	S1	S0	Valor (R$)
0	0	0	0	0	0	0	0	0	0,00
1	X	X	X	X	X	0	0	1	1,00
0	1	X	X	X	X	0	1	0	0,50
0	0	1	X	X	X	0	1	1	0,25
0	0	0	1	X	X	1	0	0	0,10
0	0	0	0	1	X	1	0	1	0,05
0	0	0	0	0	1	1	1	0	0,01

Fonte: Produzida pelos autores

Conforme a tabela, foi convencionado que a seleção para o valor de moeda X será feito na entrada X (com X variando de um a seis) do multiplexador. Portanto, para o caso de não haver operação será feito uma subtração com nada. As expressões abaixo mostram a relação das saídas pelas entradas:

$$S2 = M4 \ or \ M5 \ or \ M6$$
$$S1 = M2 \ or \ M3 \ or \ M6$$
$$S0 = M1 \ or \ M3 \ or \ M5$$

Pela lógica, a sétima entrada de seleção do multiplexador nunca irá ocorrer, mas foi convencionada como zero.

4.2.5 Verificador de Operações

Esse bloco auxiliar existe para preparar uma entrada do registrador de banco provisório ao informar que a moeda está sendo trocada e que não há nenhum erro, evitando que seja feito uma dedução da quantidade da moeda em um caso incorreto.

Para isso ele verifica se o sistema não está em erro, com banco vazio e valor de troco maior do que zero, e se a moeda está operando. Por isso, o bloco possui as entradas e saídas conforme a imagem abaixo:

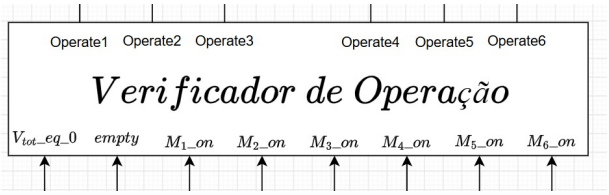


Figura 13: Verificador de Operação

Os valores das saídas de um bit são expressos em conforme a lógica em sequência:

$$Operate1 = not V_{tot}_eq_0 \ and \ empty \ and \ M_{1,on}$$
$$Operate2 = not V_{tot}_eq_0 \ and \ empty \ and \ M_{2,on}$$
$$Operate3 = not V_{tot}_eq_0 \ and \ empty \ and \ M_{3,on}$$
$$Operate4 = not V_{tot}_eq_0 \ and \ empty \ and \ M_{4,on}$$
$$Operate5 = not V_{tot}_eq_0 \ and \ empty \ and \ M_{5,on}$$
$$Operate6 = not V_{tot}_eq_0 \ and \ empty \ and \ M_{6,on}$$

4.2.6 Registradores

Um registrador é um elemento de memória digital capaz de armazenar uma palavra binária de n bits, sendo formado por n flip-flops tipo D que compartilham o mesmo sinal de clock. A cada borda ativa do clock, o registrador captura o valor presente em suas entradas de dados e o mantém em suas saídas até que uma nova captura ocorra, de modo que a saída só muda em instantes determinados pelo clock, e não imediatamente com a entrada. Para controlar quando essa captura acontece, o registrador pode possuir uma entrada de habilitação de carga: com ela em nível alto, o novo valor é armazenado na próxima borda do clock; com ela em nível baixo, o valor anterior é preservado. Além disso, uma entrada de *reset* permite levar todos os bits a um valor inicial conhecido. Dessa forma, o registrador dá memória ao circuito, permitindo guardar resultados intermediários e transferi-los entre as etapas de uma operação, o que o torna o elemento básico de um projeto em nível de transferência entre registradores.

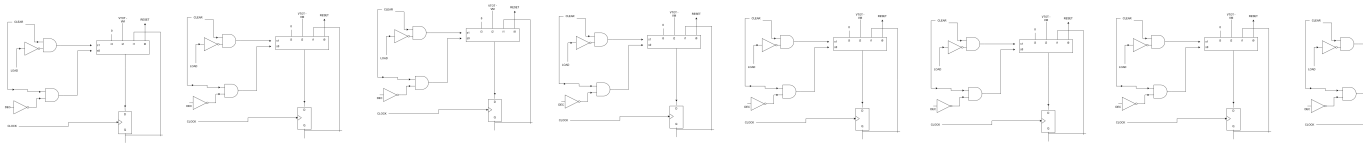


Figura 14: Registrador Vtot

Registrador Vtot O registrador V_{tot} armazena, em 10 bits, o valor total a ser tratado pela máquina de troco, sendo construído a partir de 10 fatias idênticas, uma para cada bit. Cada fatia é composta por um multiplexador 4:1 e por um flip-flop tipo D, e

todas compartilham o mesmo sinal de clock. A entrada D do flip-flop recebe a saída do multiplexador, e a saída Q do flip-flop é o próprio bit armazenado, que também é realimentado em uma das entradas do multiplexador. As quatro entradas do multiplexador correspondem às operações possíveis sobre o registrador: a entrada i_3 recebe o nível lógico 0, que zera o bit; a entrada i_2 recebe o resultado da subtração $V_{tot} - V_M$, que atualiza o valor total após a liberação de uma moeda; a entrada i_1 recebe a própria saída Q, que mantém o valor atual; e a entrada i_0 recebe o sinal *RESET*. A escolha entre essas entradas é feita pelos sinais de seleção s_1 e s_0, gerados a partir das entradas de controle *CLEAR*, *LOAD* e *DEC* por duas portas AND e dois inversores, de modo que $s_1 = CLEAR \cdot \overline{LOAD}$ e $s_0 = CLEAR \cdot \overline{DEC}$. Como esses sinais de seleção são comuns a todas as fatias, as 10 fatias executam sempre a mesma operação ao mesmo tempo, e o novo valor só é armazenado na próxima borda ativa do clock.

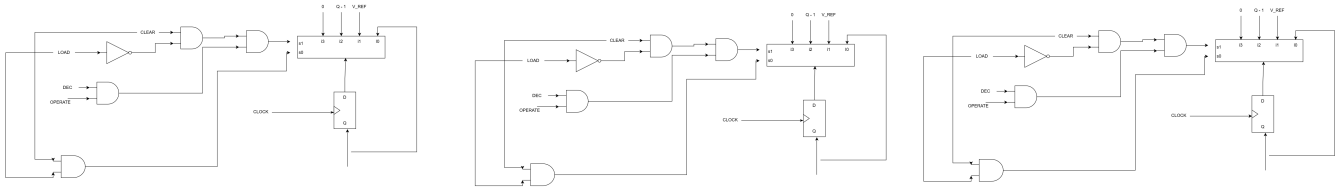


Figura 15: Registrador Banco Prvisório

Registrador Banco Provisório O registrador de valor provisório armazena um valor de 4 bits e é construído a partir de 4 fatias idênticas, uma para cada bit, todas sincronizadas pelo mesmo clock. Cada fatia é composta por um multiplexador 4:1 e por um flip-flop tipo D, cuja entrada D recebe a saída do multiplexador e cuja saída Q é o bit armazenado, que também é realimentado na entrada I_0 do multiplexador. As entradas do multiplexador correspondem às operações possíveis sobre o registrador: I_0 recebe a própria saída Q, o que mantém o valor atual; i_1 recebe V_{REF}, o que carrega o valor de referência; i_2 recebe o bit correspondente do valor decrementado $(Q - 1)$, o que decrementa o registrador; e i_3 recebe o nível lógico 0. A escolha entre essas entradas é feita pelos sinais s_1 e s_0, gerados a partir de *CLEAR*, *LOAD*, *DEC* e *OPERATE* por portas AND e por um inversor, de modo que $s_0 = CLEAR \cdot LOAD$ e $s_1 CLEAR \cdot \overline{LOAD} \cdot DEC \cdot OPERATE$. Assim, com *CLEAR* em nível alto, *LOAD* carrega V_{REF}, o decremento ocorre quando *DEC* e *OPERATE* estão ativos com *LOAD* desativado, e, em qualquer outro caso, o valor é mantido. Como os sinais de controle são comuns às 4 fatias, todos os bits executam a mesma operação ao mesmo tempo, e o novo valor só é armazenado na próxima borda ativa do clock.

Registrador Banco de Referencia O registrador de referência armazena um valor de 4 bits e é construído a partir de 4 fatias idênticas, uma para cada bit, todas sincronizadas pelo mesmo clock. Cada fatia é composta por um multiplexador 4:1 e por um flip-flop tipo

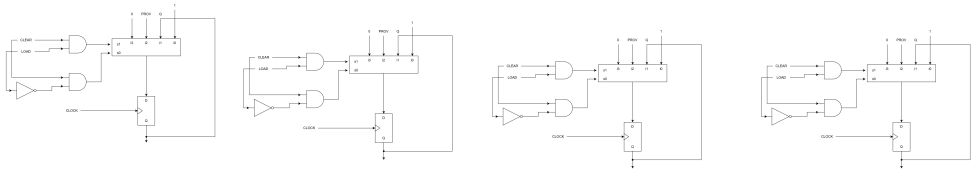


Figura 16: Banco de Referencia

D, cuja entrada D recebe a saída do multiplexador e cuja saída Q é o bit armazenado, que também é realimentado em uma das entradas do multiplexador.

As quatro entradas do multiplexador correspondem às operações possíveis sobre o registrador: i_0 recebe o nível lógico 1, o que leva o bit a 1; i_1 recebe a própria saída Q, o que mantém o valor atual; i_2 recebe o bit correspondente do valor provisório $PROV$, o que carrega esse valor no registrador; e i_3 recebe o nível lógico 0. A escolha entre essas entradas é feita pelos sinais s_1 e s_0, gerados a partir de $CLEAR$ e $LOAD$ por duas portas AND e um inversor:

$$s_1 = CLEAR \cdot LOAD, \tag{12}$$

$$s_0 = CLEAR \cdot \overline{LOAD}. \tag{13}$$

Com isso, o comportamento do registrador é o seguinte. Com $CLEAR$ em nível baixo, s_1 e s_0 valem 0 e o multiplexador seleciona i_0, de modo que todos os bits passam a 1 e o registrador assume o valor 1111. Com $CLEAR$ em nível alto e $LOAD$ em nível alto, $s_1 = 1$ e $s_0 = 0$, e o registrador carrega o valor provisório $PROV$. Com $CLEAR$ em nível alto e $LOAD$ em nível baixo, $s_1 = 0$ e $s_0 = 1$, e o valor armazenado é mantido. A combinação $s_1 s_0 = 11$ nunca ocorre, pois exigiria $LOAD$ simultaneamente em 1 e em 0, de modo que a entrada i_3 não é selecionada em nenhuma situação. Assim, a entrada D de cada flip-flop pode ser resumida por

$$D = \overline{CLEAR} + \overline{LOAD} \cdot Q + LOAD \cdot PROV. \tag{14}$$

Como os sinais $CLEAR$ e $LOAD$ são comuns às 4 fatias, todos os bits executam a mesma operação ao mesmo tempo, e o novo valor só é armazenado na próxima borda ativa do clock.

Registrador do LED de Saída das Moedas O registrador de LED é responsável por sinalizar ao usuário a situação da máquina de troco: o LED permanece piscando durante todo o tempo em que o troco está sendo processado e fica aceso de forma contínua quando ocorre um erro. Para isso, o registrador armazena o estado do LED e o atualiza por meio de um multiplexador 2:1, cujo sinal de seleção S_0 é gerado por uma lógica combinacional formada por duas portas AND e uma porta OR. A primeira porta AND combina os sinais MX_ON e $CHARGE$, que indicam que o troco está em processamento. O resultado é

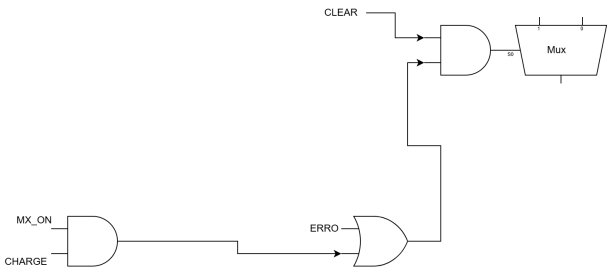


Figura 17: Registrador LED

combinado com o sinal *ERRO* por meio da porta OR, de modo que a condição se verifica tanto durante o processamento quanto na ocorrência de um erro. Por fim, a segunda porta AND combina essa condição com o sinal *CLEAR*, resultando em

$$S_0 = CLEAR \cdot \big(ERRO + MX_ON \cdot CHARGE\big).$$

O sinal *CLEAR* atua como uma habilitação geral: com *CLEAR* em nível baixo, S_0 vale 0 independentemente dos demais sinais e o multiplexador seleciona a entrada 0, mantendo o LED fora de qualquer sinalização. Com *CLEAR* em nível alto, S_0 passa a 1 sempre que há processamento em andamento ou erro, e o multiplexador seleciona a entrada 1, que define o comportamento do LED nessas situações. Como o valor selecionado é armazenado apenas na borda ativa do clock, o LED muda de estado de forma sincronizada, o que permite obter o efeito de pisca durante o processamento e o nível constante em caso de erro.

4.2.7 Relação com o Bloco de Controle

A comunicação entre o *Dataway* e a máquina de estado é feita pelos sinais apresentados na Figura 18 e descritos na Tabela 8. O *Dataway* possui seis entradas: o clock *CK* e cinco sinais de controle gerados pela máquina de estado, *LOAD_V*, *LOAD_REF*, *LOAD_PROV*, *CLEAR* e *CHANGE*. Em contrapartida, o *Dataway* fornece à máquina de estado três saídas de status: *VTOT_EQ_0*, *CHANGE_ON* e *EMPTY_BANK*.

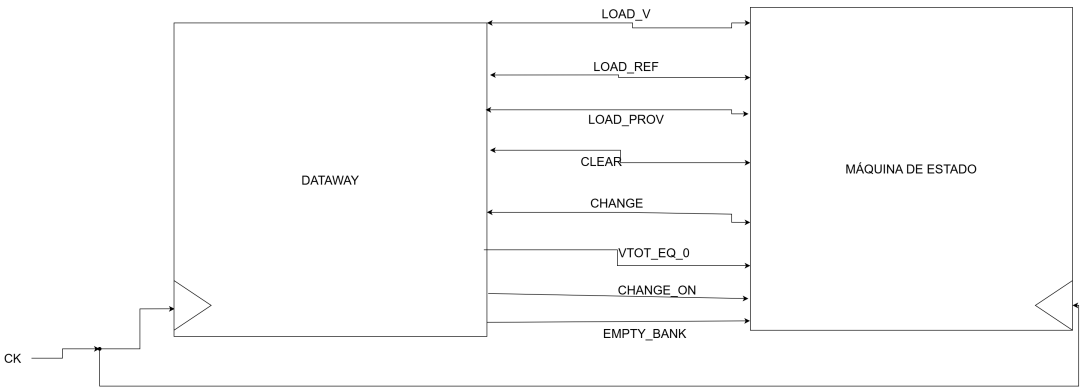


Figura 18: Interface entre o *Dataway* e a máquina de estado.

Tabela 8: Variáveis de entrada e saída do *Dataway*.

Sinal	**Tipo**	**Função**
CK	Entrada	Clock que sincroniza todos os registradores do *Dataway*.
LOAD_V	Entrada	Carrega o valor de troco V no registrador V_{tot}.
LOAD_REF	Entrada	Habilita a carga do registrador de referência (banco).
LOAD_PROV	Entrada	Habilita a carga do registrador de valor provisório.
CLEAR	Entrada	Inicializa os registradores do *Dataway*.
CHANGE	Entrada	Habilita a operação de troco, isto é, a subtração do valor da moeda selecionada.
VTOT_EQ_0	Saída	Indica que o valor restante V_{tot} é igual a zero, ou seja, que o troco foi concluído.
CHANGE_ON	Saída	Indica que a operação de troco está em andamento.
EMPTY_BANK	Saída	Indica que o banco da moeda selecionada está vazio.

Os sinais de controle *LOAD_V*, *LOAD_REF*, *LOAD_PROV*, *CLEAR* e *CHANGE* determinam quais operações o *Dataway* executa a cada borda ativa de *CK*, de modo que a máquina de estado comanda o fluxo do processo sem manipular diretamente os dados. Os sinais de status *VTOT_EQ_0*, *CHANGE_ON* e *EMPTY_BANK*, por sua vez, informam à máquina de estado o resultado das operações, e são utilizados por ela para decidir as transições entre estados: por exemplo, o fim do troco quando $V_{tot} = 0$, ou o tratamento de uma situação em que o banco da moeda selecionada esteja vazio. O clock *CK* é comum aos dois blocos, o que mantém o *Dataway* e a máquina de estado sincronizados.

4.2.8 Estrutura Geral

O dataway utiliza todos os blocos citados acima em uma instãncia cada, exceto pelos registradores de referência, provisórios e de saída, pois eles são instanciados para cada tipo de moedas, logo seis instâncias. O esquemático encontra-se na figura abaixo:

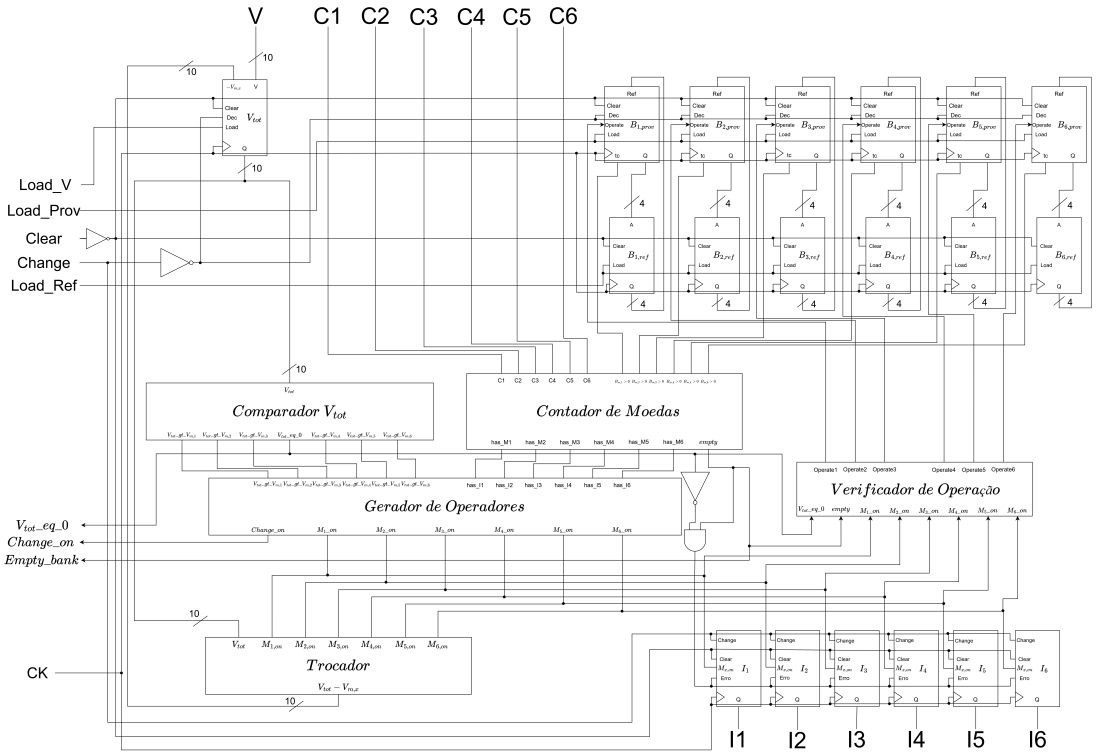


Figura 19: Esquemático completo do Dataway

Utilizando os sinais do bloco de controle e as entradas externas V, C1, C2, C3, C4, C5 e C6, o bloco de fluxo de dados envia os sinais Empty_bank, Change_on e $V_{tot}_eq_0$ para a máquina de estados e manipula os estados ligado e desligado dos leds de saída de cada bloco.

A estrutura tem como um dos objetivos principais formar os sinais que indicam qual tipo de moeda será trocado. Para isso, é o utilizado o bloco Gerador de Operadores em conjunto com o Contador de Moedas e Comparador Vtot. Com isso, o sistema obtem a informação para contar qual tipo de moeda sendo trocada no momento e utiliza esse dado para calcular o valor de Vtot após o troco e determinar qual banco provisório deve realizar o decremento de menos um.

O outro objetivo é determinar os sinais de saída do dataway para o bloco de controle. O Contador de Moedas informa o sinal de banco vazio, enquanto o Comparador Vtot diz se o sinal Vtot atual é igual a zero (troco concluído) e o Gerador de Operadores diz se está ocorrendo alguma operação de troco.

Além desses blocos, há o Verificador de Operadores que funciona para garantir que os registradores provisórios não atualizem seus valores em momentos inadequados.

O sinal de entrada do bloco de controle Clear foi barrado na sua entrada com o intuito de ajustar o sinal fornecido para funcionar corretamente com a lógica de clear dos registradores (reseta quando clear vai a zero).

O sinal change também é utilizado de forma barrada, porém somente para os registradores de valor provisório. Isso ocorre porque os registradores dos leds de saída e

os contadores decrescentes, que representam o banco de quantidades provisórias para as moedas, operam em momentos opostos para o mesmo sinal. Quando o Led deve desligar, o banco deve decrementar; quando o led deve ligar, o banco deve manter o seu valor. Com essa aplicação é possível obter um sinal pulsado nos leds para cada troco efetuado.

4.3 MDE de Baixo Nível

A máquina de estados de baixo nível para o sistema da máquina de troco converteu todos os processamentos não booleanos e operações com multiplos bits em bits singulares que modificam os valores dos registradores de múltiplas funções, que acabam por impactar na lógica interna do fluxo de dados. A estrutura que faz isso segue abaixo:

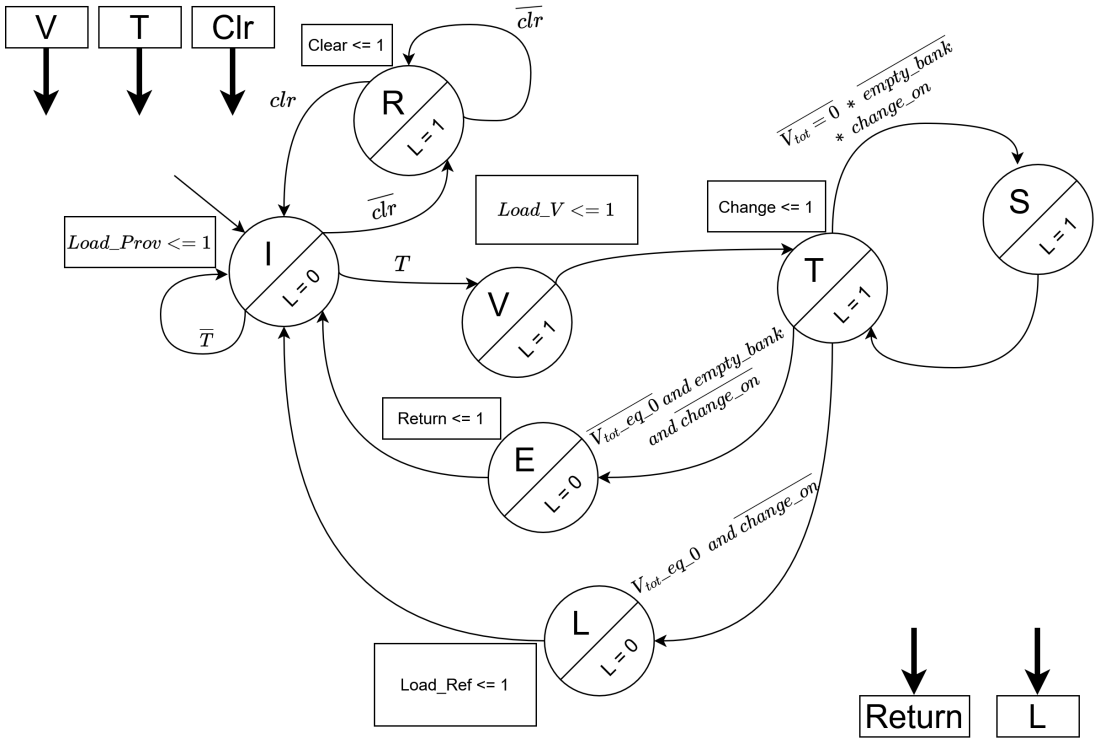


Figura 20: Esquemático da Máquina de Estados de Baixo Nível

O estado I ficou responsável por atualizar o valor do banco provisório para o valor de referência com o bit Load_Prov. Dessa forma, ao retornar do estado de erro o banco não estará mais vazio e o sinal de erro não estará ativo, assim os leds que ficaram ligados devido ao sinal de erro alto irão desligar.

O estado V ficou responsável por acionar o bit que atualiza o valor de V para Vtot, podendo iniciar os cálculos para o troco. Enquanto que o estado R ficou de ligar Clear para resetar os registradores (não confunda a chave clr com o sinal de controle Clear).

O estado T liga a bit de change para ligar o led adequado, enquanto que o estado S agora só precisa desligar todas as 5 variáveis, devido ao barramento interno do sinal Change no Dataway para os registradores do banco provisório.

O estado de erro E só precisa ligar a saída externa Return, pois zerando as outras variáveis já impede que o sistema não transfira as moedasde troco incompleto para o usuário. Enquanto que o estado L aciona o bit Load_Ref para atualizar o banco de referência.

Conclusão

O desenvolvimento da máquina de troco permitiu consolidar os conhecimentos teóricos da disciplina de Sistemas Digitais por meio de uma aplicação prática voltada ao projeto RTL. O sistema foi estruturado em blocos, incluindo registradores para armazenar valores de moedas, total acumulado e troco a ser fornecido, além da unidade de controle principal, o que facilitou a validação de cada bloco e a integração em um sistema completo. Foram utilizados elementos combinacionais, como somadores e multiplexadores, e sequenciais, como Flip-Flops do tipo D. Além disso, foi aplicado o projeto RTL.

REFERÊNCIAS

TOCCI, R.; WIDMER, N.; MOSS, G. *Digital Systems: Principles and Applications*. [S.l.]: Pearson Education Limited, 2011. ISBN 9780130387936.

VAHID, F. *Sistemas Digitais*. [S.l.]: Grupo A - Bookman, 2009. ISBN 9788577802371.

APÊNDICE A - Relato semanal

A.1 Equipe

Tabela 1 - Identificação da equipe.

Função no grupo:	Nome completo do aluno	Responsável por
Líder:	Francisco Wellyson	Relatório
Redator:	Mateus Macedo	Relatório
Debatedor:	Aysha Dandara	Relatório

Fonte: Produzido pelos autores

A.2 Defina o problema

Projeto da Máquina de troco

A.3 Registro do *brainstorming*

No dia que foi passado o projeto tivemos conversa sobre como seria feito a Máquina de Estado de alto nível

A.4 Pontos-chaves

Projeto RTL, Máquina de estados de alto nível

A.5 Questões de pesquisa

Máquina de estado de alto nível, Projetos RTL, Registradores

A.6 Planejamento da pesquisa

VAHID, F. *Sistemas Digitais.* [S.l.]: Grupo A - Bookman, 2009. ISBN 9788577802371. e TOCCI, R.; WIDMER, N.; MOSS, G. *Digital Systems: Principles and Applications.* [S.l.]: Pearson Education Limited, 2011. ISBN 9780130387936.